

Transferencia de
datos,
direccionamiento
y aritmética

Se destacará una diferencia fundamental entre el lenguaje ensamblador y los lenguajes de alto nivel. En el lenguaje ensamblador, es necesario preocuparse por el almacenamiento de datos y los detalles específicos de la máquina. Los compiladores de lenguajes de alto nivel como C++ y Java realizan una estricta comprobación de tipos en las variables y las asignaciones para evitar errores lógicos relacionados con datos de tipos incorrectos. En contraste, los ensambladores brindan una gran libertad al declarar y mover datos, realizando poca comprobación de errores y ofreciendo una variedad de operadores y expresiones de direcciones. Sin embargo, esta libertad viene con el costo de tener que dominar muchos detalles para escribir programas significativos.

Tipos de operandos

El lenguaje ensamblador existen 3 tipos de operandos de instrucciones: inmediatos, de registros y de memoria. De los 3, solo el tercero es un poco complicado. La siguiente tabla presenta una notación simple para los operandos, adaptada de los manuales de la familia y ha guion 32 de Intel.

Operando	Descripción
<i>r8</i>	Registro de propósito general de 8 bits: AH, AL, BH, BL, CH, CL, DH, DL
<i>r16</i>	Registro de propósito general de 16 bits: AX, BX, CX, DX, SI, DI, SP, BP
<i>r32</i>	Registro de propósito general de 32 bits: EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP
<i>reg</i>	Cualquier registro de propósito general
<i>sreg</i>	Registro de segmento de 16 bits: CS, DS, SS, ES, FS, GS
<i>imm</i>	Valor inmediato de 8, 16 o 32 bits
<i>imm8</i>	Valor tipo byte inmediato de 8 bits
<i>imm16</i>	Valor tipo palabra inmediato de 16 bits
<i>imm32</i>	Valor tipo doble palabra inmediato de 32 bits
<i>r/m8</i>	Operando de 8 bits, que puede ser un registro general de 8 bits o un byte de memoria
<i>r/m16</i>	Operando de 16 bits, que puede ser un registro general de 16 bits o una palabra de memoria
<i>r/m32</i>	Operando de 32 bits, que puede ser un registro general de 32 bits o una doble palabra de memoria
<i>mem</i>	Cualquier operando de memoria de 8, 16 o 32 bits

Operandos directos de memoria

Anteriormente explicamos que los nombres de las variables son referencias a desplazamientos dentro del segmento de datos. Por ejemplo, la siguiente declaración indica que se ha asignado al segmento de datos un byte que contiene el número 10h:

```
.data  
var1 BYTE 10h
```

El código del programa contiene instrucciones que emplean (buscan) operandos que hacen referencia a direcciones de memoria. Suponga que **var1** se ubicó en el desplazamiento 10400h. Una instrucción en lenguaje ensamblador para mover esta variable al registro AL podría ser:

```
mov    AL, var1
```

MASM ensamblaría esta instrucción como el siguiente código de máquina:

A0 00010400

El primer byte en la instrucción de máquina es el código de operación.

La parte restante es la dirección hexadecimal de 32 bits de **var1**.

Aunque se podrían escribir programas sólo con direcciones numéricas, los nombres simbólicos como **var1** facilitan el proceso de referenciar la memoria.

Notación alternativa. Algunos programadores prefieren usar la siguiente notación con operandos directos, ya que los corchetes implican una operación que emplean operandos que hacen referencia a una dirección de memoria:

```
mov  al, [var1]
```

MASM permite esta notación, por lo que usted puede utilizarla en sus propios programas, si así lo desea. Debido a que muchos programas (incluyendo los de Microsoft) se imprimen sin los corchetes, sólo los utilizaremos en este libro cuando esté involucrada una expresión aritmética:

```
mov  al, [var1 + 5]
```

Instrucción mov

La instrucción MOV copia datos de un operando de origen a un operando de destino. Esta instrucción, conocida como *transferencia de datos*, se utiliza en casi todos los programas. Su formato básico muestra que el primero operando es el destino y el segundo es el origen:

`MOV destino,origen`

El contenido del operando de destino cambia, pero el del operando de origen no. El movimiento de datos de derecha a izquierda es similar a la instrucción de asignación en C++ o Java:

`destino = origen;`

En casi todas las instrucciones en lenguaje ensamblador, el operando izquierdo es el destino y el operando derecho es el origen

Los siguientes ejemplos utilizan registros para todos los operandos, mostrando todas las variaciones de tamaño:

```
mov    bx,0A69Bh
```

```
movzx  eax,bx           ; EAX = 0000A69Bh
```

```
movzx  edx,bl           ; EDX = 0000009Bh
```

```
movzx  cx,bl            ; CX = 009Bh
```

MOV es bastante flexible en el uso de sus operandos, siempre y cuando se observen las siguientes reglas:

- Ambos operandos deben ser del mismo tamaño.
- Ambos operandos no pueden ser operandos de memoria.
- CS, EIP e IP no pueden ser operandos de destino.
- Un valor inmediato no puede moverse a un registro de segmento.

He aquí una lista de las variantes generales de MOV, excluyendo los registros de segmento:

MOV reg,reg

MOV mem,reg

MOV reg,mem

MOV mem,imm

MOV reg,imm

Memoria a memoria Una sola instrucción MOV no puede usarse para mover datos directamente de una ubicación de memoria a otra. En vez de ello, puede mover el valor del operando de origen a un registro, antes de mover su valor a un operando de memoria:

```
.data  
var1 WORD ?  
var2 WORD ?  
.code  
mov ax,var1  
mov var2,ax
```

Instrucción movzx

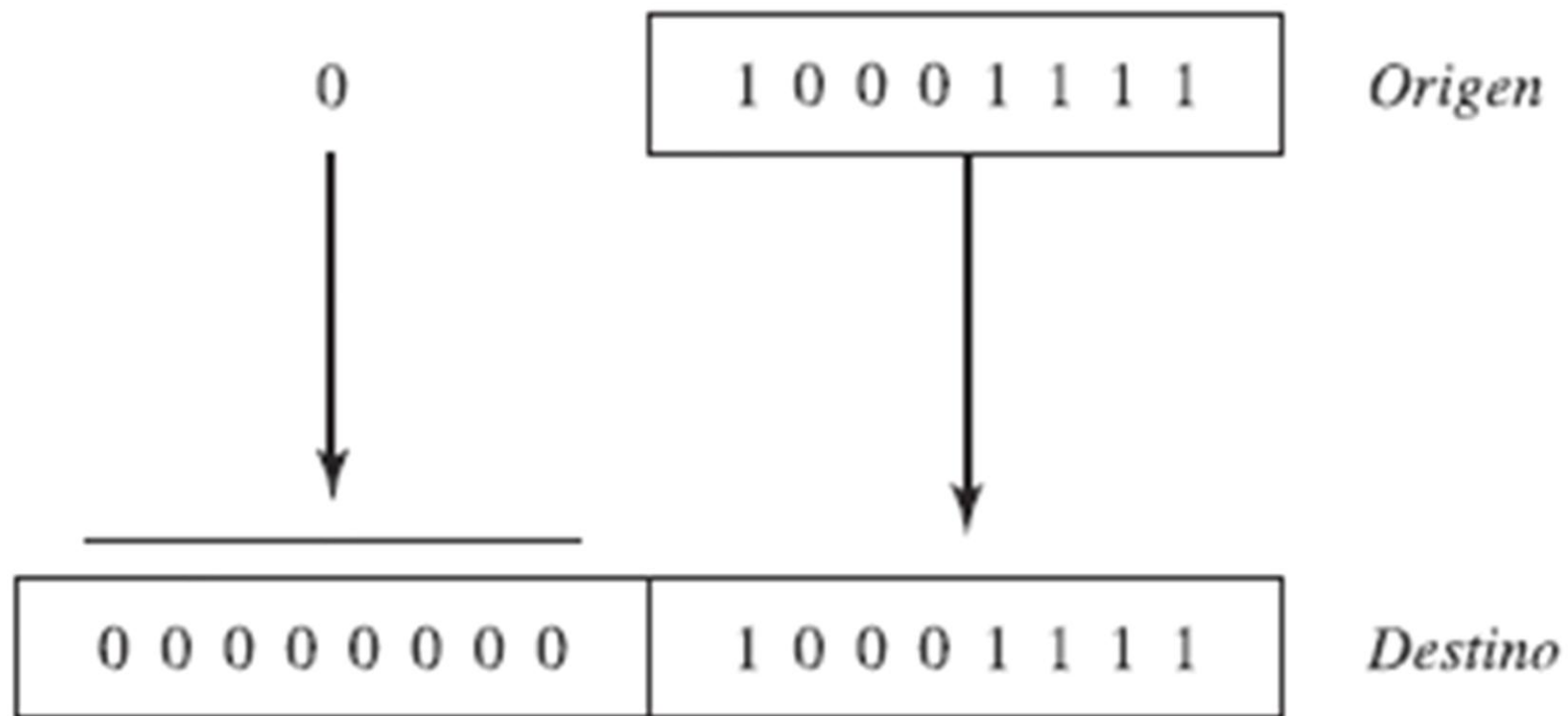
La instrucción MOVZX (*mover con extensión de ceros*) copia el contenido de un operando de origen a un operando de destino, y extiende con ceros el valor hasta 16 o 32 bits. Esta instrucción se utiliza sólo con enteros sin signo. Hay tres variantes:

```
MOVZX    r32, r/m8  
MOVZX    r32, r/m16  
MOVZX    r16, r/m8
```

En cada una de las tres variantes, el primer operando (un registro) es el destino y el segundo es el origen. La siguiente instrucción mueve el número binario 10001111 a AX:

```
movzx ax,10001111b
```

Diagrama de MOVZX ax, 8Fh.



```
.data
byte1  BYTE  9Bh
palab1 WORD  0A69Bh
.code
movzx  eax,palab1          ; EAX = 0000A69Bh
movzx  edx,byte1           ; EDX = 0000009Bh
movzx  cx,byte1            ; CX  = 009Bh
```

Los siguientes ejemplos utilizan operandos de memoria para el origen y producen los mismos resultados:

Instrucción movsx

La instrucción MOVSEX (mover con extensión de signo) copia el contenido de un operando de origen en un operando de destino, y extiende con signo el valor hasta 16 o 32 bits. Esta instrucción sólo se utiliza con enteros con signo. Existen tres variantes:

MOVSEX *r32*, *r/m8*

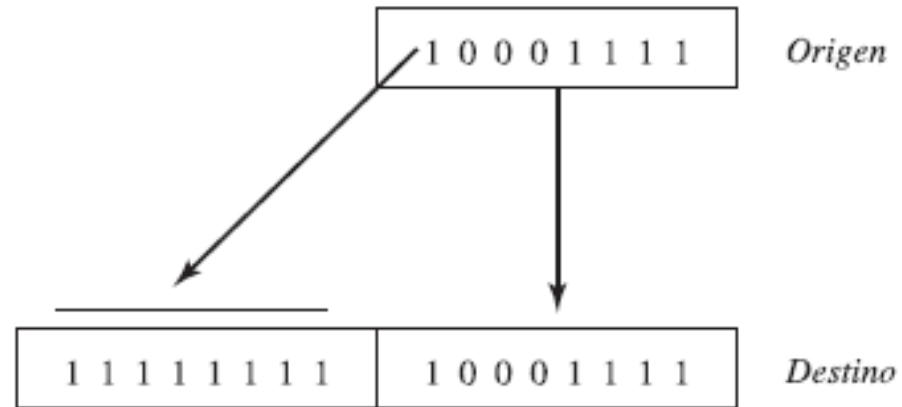
MOVSEX *r32*, *r/m16*

MOVSEX *r16*, *r/m8*

Para extender con signo un operando, se toma el bit más alto del operando más pequeño y se repite (replica) este bit a lo largo de los bits extendidos en el operando de destino. Suponga que el valor de 8 bits 10001111b se mueve a un destino de 16 bits:

movsx ax,10001111b

Diagrama de MOVSX ax,10001111b.



El bit superior del origen se copia en cada una de las posiciones de los 8 bits superiores del destino.

He aquí unos cuantos ejemplos más que utilizan una variedad de tamaños de registros:

```
mov     bx,0A69Bh
movsx   eax,bx           ; EAX = FFFFA69Bh
movsx   edx,bl           ; EDX = FFFFFFF9Bh
movsx   cx,bl            ; CX  = FF9Bh
```

Instrucciones xchg

La instrucción XCHG (intercambiar datos) intercambia el contenido de dos operandos. Hay tres variantes:

```
XCHG    reg, reg  
XCHG    reg, mem  
XCHG    mem, reg
```

Las reglas para los operandos en la instrucción XCHG son las mismas que para la instrucción MOV, excepto que XCHG no acepta operandos inmediatos. En las aplicaciones para ordenar arreglos, XCHG proporciona una manera simple de intercambiar los elementos de dos arreglos. He aquí unos cuantos ejemplos del uso de XCHG:

xchg	ax,bx	; intercambia registros de 16 bits
xchg	ah,al	; intercambia registros de 8 bits
xchg	var1,bx	; intercambia op mem de 16 bits con BX
xchg	eax,ebx	; intercambia registros de 32 bits

Para intercambiar dos operandos de memoria, utilice un registro como contenedor temporal y combine MOV con XCHG:

```
mov    ax,val1
xchg   ax,val2
mov    val1,ax
```

Operandos de desplazamiento directo

Para crear un operando de desplazamiento directo podemos sumar un desplazamiento al nombre de una variable. Esto nos permite acceder a ubicaciones de memoria que tal vez no tengan etiquetas explícitas. Empecemos con un arreglo de bytes llamado **arregloB**:

```
arregloB  BYTE 10h,20h,30h,40h,50h
```

Si utilizamos MOV con **arregloB** como el operando de origen, moveremos de manera automática el primer byte en el arreglo:

```
mov al,arregloB ; AL = 10h
```

Podemos acceder al segundo byte en el arreglo, sumando 1 al desplazamiento de **arregloB**:

```
mov al,[arregloB+1] ; AL = 20h
```

Para acceder al tercer byte, le sumamos 2:

```
mov al,[arregloB+2] ; AL = 30h
```

Una expresión como **arregloB1** produce lo que se conoce como *dirección efectiva*, al sumar una constante al desplazamiento de la variable. Al encerrar una dirección efectiva entre corchetes, indicamos que la expresión se utiliza para hacer referencia a una dirección de memoria para obtener su contenido. MASM no requiere los corchetes, por lo que las siguientes instrucciones son equivalentes:

```
mov al,[arregloB+2]
```

```
mov al,arregloB+2
```

Comprobación de rango MASM no tiene comprobación de rango integrada para las direcciones efectivas. Si ejecutamos la siguiente instrucción, el ensamblador sólo obtiene un byte de memoria fuera del arreglo. El resultado es un traicionero error lógico, por lo que debemos ser muy cuidadosos al comprobar las referencias a arreglos:

```
mov al,[arregloB+20] ; AL = ??
```

Arreglos tipo palabra y doble palabra En un arreglo de palabras de 16 bits, el desplazamiento de cada elemento del arreglo es 2 bytes más adelante del anterior. Ésta es la razón por la cual sumamos 2 a **arregloW** en el siguiente ejemplo, para llegar al segundo elemento:

```
.data
arregloW WORD 100h,200h,300h
.code
mov ax,arregloW           ; AX = 100h
mov ax,[arregloW+2]       ; AX = 200h
```

De manera similar, el segundo elemento en un arreglo tipo doble palabra se encuentra a 4 bytes más adelante del primero:

```
data
arregloD DWORD 10000h,20000h
.code
mov eax,arregloD          ; EAX = 10000h
mov eax,[arregloD+4]      ; EAX = 20000h
```

```

.386
.model flat,stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:DWORD
DumpRegs PROTO

.data
val1 WORD 1000h
val2 WORD 2000h
arregloB BYTE 10h,20h,30h,40h,50h
arregloW WORD 100h,200h,300h
arregloD DWORD 10000h,20000h

.code
main PROC

; MOVZX
mov bx,0A69Bh
movzx eax,bx ; EAX = 0000A69Bh
movzx edx,bl ; EDX = 0000009Bh
movzx cx,bl ; CX = 009Bh
; MOVSX
mov bx,0A69Bh
movsx eax,bx ; EAX = FFFFA69Bh
movsx edx,bl ; EDX = FFFFFFF9Bh
mov bl,7Bh
movsx cx,bl ; CX = 007Bh
; Intercambio de memoria a memoria:
mov ax,val1 ; AX = 1000h
xchg ax,val2 ; AX = 2000h, val2 = 1000h
mov val1,ax ; val1 = 2000h
; Direcccionamiento con desplazamiento directo (arreglo de bytes):
mov al,arregloB ; AL = 10h
mov al,[arregloB+1] ; AL = 20h
mov al,[arregloB+2] ; AL = 30h
; Direcccionamiento con desplazamiento directo (arreglo de palabras):
mov ax,arregloW ; AX = 100h
mov ax,[arregloW+2] ; AX = 200h
; Direcccionamiento con desplazamiento directo (arreglo de dobles palabras):
mov eax,arregloD ; EAX = 10000h
mov eax,[arregloD+4] ; EAX = 20000h
mov eax,[arregloD+TYPE arregloD] ; EAX = 20000h

main ENDP
END main

```

Cuestionario

